

IT3101 – Data Warehousing and Business Intelligence

**Sri Lanka Institute of Information
Technology**



**Design and Implementation of a Data Warehouse and Business
Intelligence Solution**

2026-DS-51

Student ID	Name
IT24100738	D.D.E Udawaththa
IT24100638	Pahasara H. M. P
IT24100546	Navodya G.L.T
IT24100244	Nakkavita N.K.P.P

B.Sc. (Hons) in Information Technology

Specializing in Data Science

Table of contents

<i>AI Usage Acknowledgement</i>	4
<i>Task 1: Dataset Selection and Business Scenario Identification</i>	5
1.1 Business Domain	5
1.2 Purpose of the System	5
1.3 Business Problem Addressed	5
1.4 Dataset Source	5
1.5 Dataset Format	5
1.6 Number of Records	5
1.7 Attribute Description	6
1.8 Why This Dataset Is Suitable	7
<i>Task 2: Data Source Identification and Preparation</i>	8
2.1 Description of Each Data Source	8
2.2 Source Format	8
2.3 Available Attributes	8
2.4 Relationship Between Sources	8
2.5 Data Preparation Steps	8
a) Encoding Check	8
b) Category Translation Validation	9
c) Date Column Conversion	9
d) Referential Integrity Check	9
e) Duplicate Detection	9
f) Multi-Format Source Creation	9
g) Relational Database Creation	9
2.6 Integration Explanation	10
<i>Task 3: Data Warehouse Architecture Design</i>	11
<i>Task 4: Dimensional Data Warehouse Design and Implementation</i>	13
4.1 Fact Table Definition	13
4.2 Dimension Tables	13
4.3 Schema Design	14

4.4 Implementation	15
4.5 Design Assumptions	15
<i>Task 5 - ETL Process Development</i>	17
5.1 Extract	17
5.2 Transform	17
5.3 Load	18
<i>Task 6: Data Mart Development</i>	21
6.1 Purpose of the Data Mart	21
6.2 Target Users	21
6.3 Analytical Benefits	21
6.4 Implementation	21
<i>Task 7: OLAP Analysis and Business Intelligence Dashboard Development</i>	22
Report Page 1: Executive Summary	22
Report Page 2: Logistics and Delivery Performance	23
Report Page 3: Payment Mix and Installments	24
Summary of OLAP Operations	25
<i>Task 8: Business Insights and Recommendations</i>	26
<i>References</i>	28
<i>Appendix A - Attribute Summary</i>	29
<i>Appendix B - Data Warehouse Schema (SQLite, built with Python/pandas)</i>	31
<i>Appendix C — Equivalent SQL Server Implementation</i>	33
C.1 — Star Schema Creation (03_create_star_schema.sql)	33
C.2 — ETL Load Procedures (04_etl_load_dw.sql)	35

AI Usage Acknowledgement

AI tools (Claude, by Anthropic) were used as a supporting resource during the completion of this assignment, in line with the module's AI usage policy. Specific uses included:

- **Brainstorming and planning:** discussing and refining the choice of dataset, business scenario, and overall project structure across the eight tasks.
- **Code assistance:** generating and debugging Python (pandas) scripts for data profiling and source preparation (Task 2), and T-SQL scripts for staging table creation, and ETL transformation logic (Task 5). All scripts were run, tested, and verified against the actual dataset by the author(s); errors encountered during execution were debugged with AI assistance and corrected before use.
- **Documentation support:** assistance structuring report sections and phrasing explanations for architecture, schema design, and business insights, based on the author's own implementation, screenshots, and dashboard outputs.
- **Diagram generation:** the architecture diagram (Task 3) was generated with AI assistance based on the author's own design decisions, then reviewed for accuracy against the implemented database.

All final SQL scripts, database implementation, ETL execution, Power BI dashboard, and business insights were independently run, tested, and validated by the authors using their own dataset and environment (SSMS / VS Code / Power BI). AI-generated content was critically reviewed, verified against actual query outputs, and adapted where necessary rather than used unchanged. The analytical findings and interpretations presented in Task 8 are based on the author's own review of the dashboard outputs.

Task 1: Dataset Selection and Business Scenario Identification

1.1 Business Domain

The selected dataset belongs to the Retail / E-commerce domain. It represents a Brazilian online marketplace that connects independent sellers with customers across the country.

1.2 Purpose of the System

The source system is an operational (OLTP) platform that processes customer orders, tracks individual order items, manages sellers and products, records payments, and captures post-purchase customer reviews. It supports the day-to-day transactional operations of an e-commerce marketplace.

1.3 Business Problem Addressed

Management cannot easily determine which regions, product categories, and sellers generate the most revenue, or how delivery delays affect customer satisfaction, because this information is spread across multiple separate operational files rather than being available in a unified analytical structure. A data warehouse and BI solution is needed to consolidate this data and support decision-making.

1.4 Dataset Source

- **Name:** Olist Brazilian E-Commerce Public Dataset[1]
- **Source:** Kaggle (<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>)
- **Time period covered:** 2016–2018

1.5 Dataset Format

The dataset was sourced and prepared using multiple formats to represent a realistic multi-system operational environment:

- CSV files (transactional data: orders, order items, payments, reviews)
- Relational database (master data: customers, products, sellers, geolocation, loaded into SQLite)
- Excel spreadsheet (category translation lookup)
- JSON via REST API (public holiday reference data)
- XML file (category translation lookup, alternate format)

1.6 Number of Records

Source File	Record Count
Orders	99,441
Order Items	112,650
Customers	99,441
Products	32,951
Sellers	3,095

Payments	103,886
Reviews	99,224
Geolocation	1,000,163
Category Translation	71

1.7 Attribute Description

The table below summarises the attributes of each source table. A full attribute-level listing, including data types and null percentages, is provided in Appendix A (exported as task1_attribute_summary.csv).

Table	Key Attributes	Description
Orders	order_id, customer_id, order_status, purchase/delivery timestamps	Core transaction record
Order Items	order_id, product_id, seller_id, price, freight_value	Line-item detail per order
Customers	customer_id, customer_unique_id, city, state	Customer master data
Products	product_id, product_category_name, weight, dimensions	Product master data
Sellers	seller_id, city, state	Seller master data
Payments	order_id, payment_type, installments, payment_value	Payment detail per order
Reviews	order_id, review_score, review dates	Customer satisfaction data
Geolocation	zip_code_prefix, lat, lng, city, state	Location reference data

--- orders (99,441 records, 8 attributes) ---

	dtype	non_null_count	null_count	null_pct	sample_value
order_id	object	99441	0	0.00	e481f51cbdc54678b7cc49136f2d6af7
customer_id	object	99441	0	0.00	9ef432eb6251297304e76186b10a928d
order_status	object	99441	0	0.00	delivered
order_purchase_timestamp	object	99441	0	0.00	2017-10-02 10:56:33
order_approved_at	object	99281	160	0.16	2017-10-02 11:07:15
order_delivered_carrier_date	object	97658	1783	1.79	2017-10-04 19:55:00
order_delivered_customer_date	object	96476	2965	2.98	2017-10-10 21:25:13
order_estimated_delivery_date	object	99441	0	0.00	2017-10-18 00:00:00

--- order_items (112,650 records, 7 attributes) ---

	dtype	non_null_count	null_count	null_pct	sample_value
order_id	object	112650	0	0.0	00010242fe8c5a6d1ba2dd792cb16214
order_item_id	int64	112650	0	0.0	1
product_id	object	112650	0	0.0	4244733e06e7ecb4970a6e2683c13e61
seller_id	object	112650	0	0.0	48436dade18ac8b2bce089ec2a041202
shipping_limit_date	object	112650	0	0.0	2017-09-19 09:45:35
price	float64	112650	0	0.0	58.9
freight_value	float64	112650	0	0.0	13.29

1.8 Why This Dataset Is Suitable

This dataset represents a genuine OLTP system with multiple related transactional and master-data tables. It contains sufficient volume for meaningful analytical processing, includes attributes suitable for building customer, product, seller, location, and date dimensions, and supports the construction of a realistic sales fact table at the order-item grain. Its relational structure and real-world business context make it well suited to developing an end-to-end Data Warehouse and BI solution.

Task 2: Data Source Identification and Preparation

2.1 Description of Each Data Source

Source	Format	Contents
Orders, Order Items, Payments, Reviews	CSV	Transactional business events
Customers, Products, Sellers, Geolocation	Relational database (SQLite)	Master and reference data
Category Translation	Excel (.xlsx)	Portuguese-to-English category lookup
Public Holidays	JSON (via REST API) [2]	Reference data supporting DimDate
Category Translation	XML	Alternate-format Portuguese-to-English category lookup

2.2 Source Format

Each source was read using an appropriate method: CSV files were loaded with pandas `read_csv`; relational master data was queried using a SQLite connection; the Excel lookup file was read with pandas `read_excel`; and holiday data was retrieved using the requests library and parsed from JSON. The XML version of the category translation lookup was read using Python's XML parsing library.

2.3 Available Attributes

Full attribute listings for each source are provided in Section 1.7 and Appendix A.

2.4 Relationship Between Sources

The sources are linked through the following key relationships:

- `orders.order_id` → `order_items.order_id`, `payments.order_id`, `reviews.order_id`
- `orders.customer_id` → `customers.customer_id`
- `order_items.product_id` → `products.product_id`
- `order_items.seller_id` → `sellers.seller_id`
- `products.product_category_name` → `category_translation.product_category_name`
- `customers/sellers zip code prefix` → `geolocation.geolocation_zip_code_prefix`

2.5 Data Preparation Steps

a) Encoding Check

Verified that Portuguese accented characters in city names (e.g. "São Paulo") loaded correctly under UTF-8 encoding, avoiding garbled text in downstream reporting.

b) Category Translation Validation

Cross-checked all product categories in the products table (73 distinct categories) against the translation lookup table (71 categories). Found 2 categories with no matching English translation: `pc_gamer` and `portateis_cozinha_e_preparadores_de_alimentos`. Decision: these two categories were manually translated (`pc_gamer` → `pc_gamer`; `portateis_cozinha_e_preparadores_de_alimentos` → `portable_kitchen_food_preparers`) rather than defaulted to "Other", since both represent valid, identifiable product categories.

c) Date Column Conversion

All date fields in the orders table were originally stored as text strings. Five date columns were identified and converted to proper datetime format: `order_purchase_timestamp` (0% null), `order_approved_at` (0.16% null), `order_delivered_carrier_date` (1.79% null), `order_delivered_customer_date` (2.98% null), and `order_estimated_delivery_date` (0% null). The null values in `order_delivered_customer_date` correspond to orders that were cancelled or not yet delivered, and this has been flagged for handling in the ETL stage.

d) Referential Integrity Check

Verified consistency of key fields across separately sourced files. Found 0 unmatched `order_id` values, 0 unmatched `customer_id` values, 0 unmatched `product_id` values, and 0 unmatched `seller_id` values between related tables. This confirms the sources can be reliably joined during integration.

e) Duplicate Detection

The geolocation source contains 1,000,163 rows but only 19,015 distinct zip code prefixes, indicating a duplication rate of approximately 98.1%. Decision: duplicate rows will be resolved by averaging latitude and longitude per zip prefix during the ETL Transform stage.

f) Multi-Format Source Creation

To represent a realistic multi-system source environment, the category translation lookup was converted into Excel and XML format, and 2016–2018 Brazilian public holiday data (36 holidays) was retrieved live from a public REST API (Nager.Date)[2] and saved in JSON format.

g) Relational Database Creation

Customer, product, seller, and geolocation master data were loaded into a SQLite relational database (`olist_source.db`) to represent the relational OLTP source system component of the architecture.

2.6 Integration Explanation

The five source types used - flat files (CSV), a spreadsheet (Excel), an XML file, a relational database (SQLite), and an external API (JSON) - together represent a realistic multi-system operational environment typical of a real e-commerce business. All sources share common key fields, including order_id, customer_id, product_id, seller_id, and zip code prefix, which enables them to be reliably integrated into a single, unified analytical structure during the ETL process described in Task 5, following the cleaning and validation steps documented above.

Task 3: Data Warehouse Architecture Design

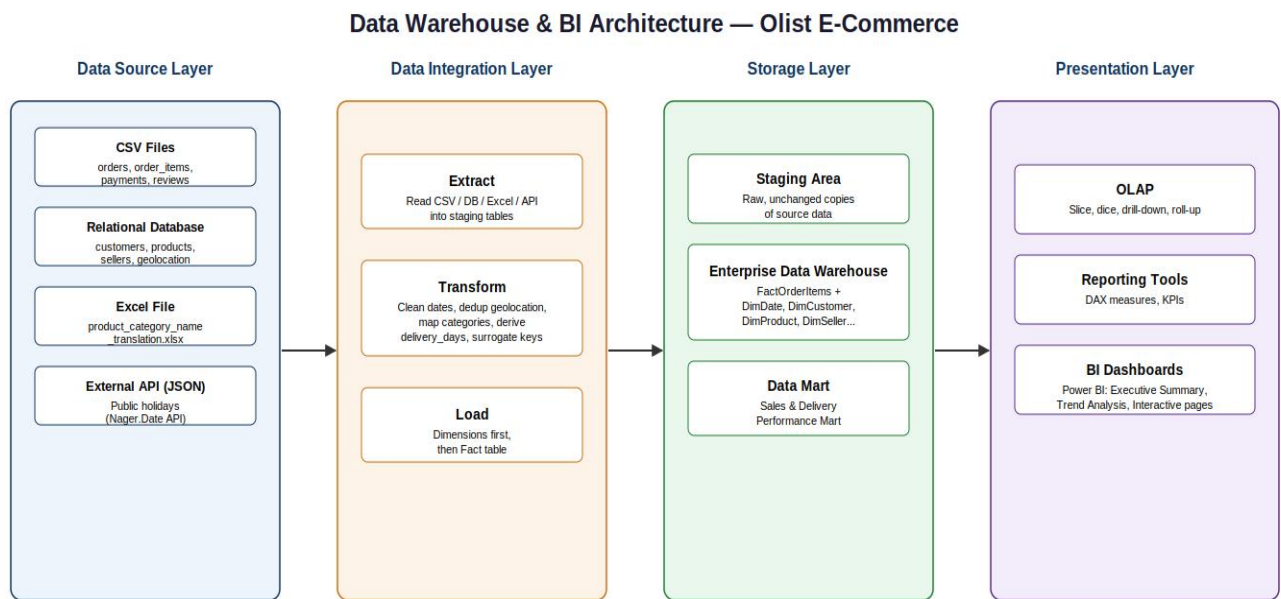


Figure: End-to-end DW/BI architecture — Source → Integration → Storage → Presentation

Explanation of each layer

1. Data Source Layer — This is where the raw operational data lives. In this project it includes 8 CSV files (customers, orders, order items, payments, reviews, products, sellers, geolocation), one Excel file (category translation), one JSON source from a public holiday API, and one SQLite relational database holding the master data tables. Together these represent the different types of source systems a real business would have.

2. Data Integration Layer — This layer moves data from the sources into the warehouse. It performs three steps:

- **Extract** — pull data from the CSVs, Excel file, API, and database
- **Transform** — clean, standardize and reshape it (handling missing values, converting data types, translating categories, de-duplicating geolocation)
- **Load** — write the cleaned data into the warehouse tables

3. Storage Layer — Has three parts:

- **Staging area** — a temporary holding area for raw/lightly-cleaned data before it's transformed.
- **Enterprise Data Warehouse** — the final star-schema warehouse holding fact and dimension tables.
- **Data Mart** — a smaller, subject-specific slice of the warehouse (in this project, a Sales data mart) built for a specific department or use case.

4. Presentation Layer — Where business users interact with the data:

- **OLAP** — multi-dimensional analysis (drill-down, roll-up, slice, dice)
- **Reporting tools / BI dashboards** — Power BI dashboards showing KPIs, trends, and interactive filters.

How data flows through it: raw files and API/database sources enter the Data Source layer → get processed by a Python ETL pipeline in the Integration layer → land in the Storage layer as a clean star schema in a SQLite warehouse → get visualized in a BI dashboard in the Presentation layer for business decision-making.

Task 4: Dimensional Data Warehouse Design and Implementation

4.1 Fact Table Definition

This project uses two fact tables, since payment data is recorded at the order level in the source system, not per item, and forcing it into one table would distort totals.

FactSales

- **Business process:** Online order fulfilment (sales)
- **Grain:** One row per order item — one product line within one customer order
- **Measures:** price, freight_value, delivery_days, review_score
- **Foreign keys:** PurchaseDateKey, CustomerKey, ProductKey, SellerKey, CustomerGeoKey, PaymentTypeKey, OrderStatusKey

FactPayment

- **Business process:** Payment collection Grain: One row per payment
- **Measures:** payment_value, payment_installments
- **Foreign keys:** PurchaseDateKey, CustomerKey, PaymentTypeKey

4.2 Dimension Tables

Dimension	Attributes	Hierarchy
DimDate	full_date, day, month, quarter, year, is_weekend, is_holiday	Day → Month → Quarter → Year
DimCustomer	customer_id, customer_unique_id, city, state, region	City → State → Region
DimProduct	product_id, category (PT/EN), weight, dimensions	Category
DimSeller	seller_id, city, state, region	City → State → Region
DimPaymentType	payment_type	-
DimGeolocation	zip code prefix, latitude, longitude, region	-
DimOrderStatus	order_status, is_delivered flag	

4.3 Schema Design

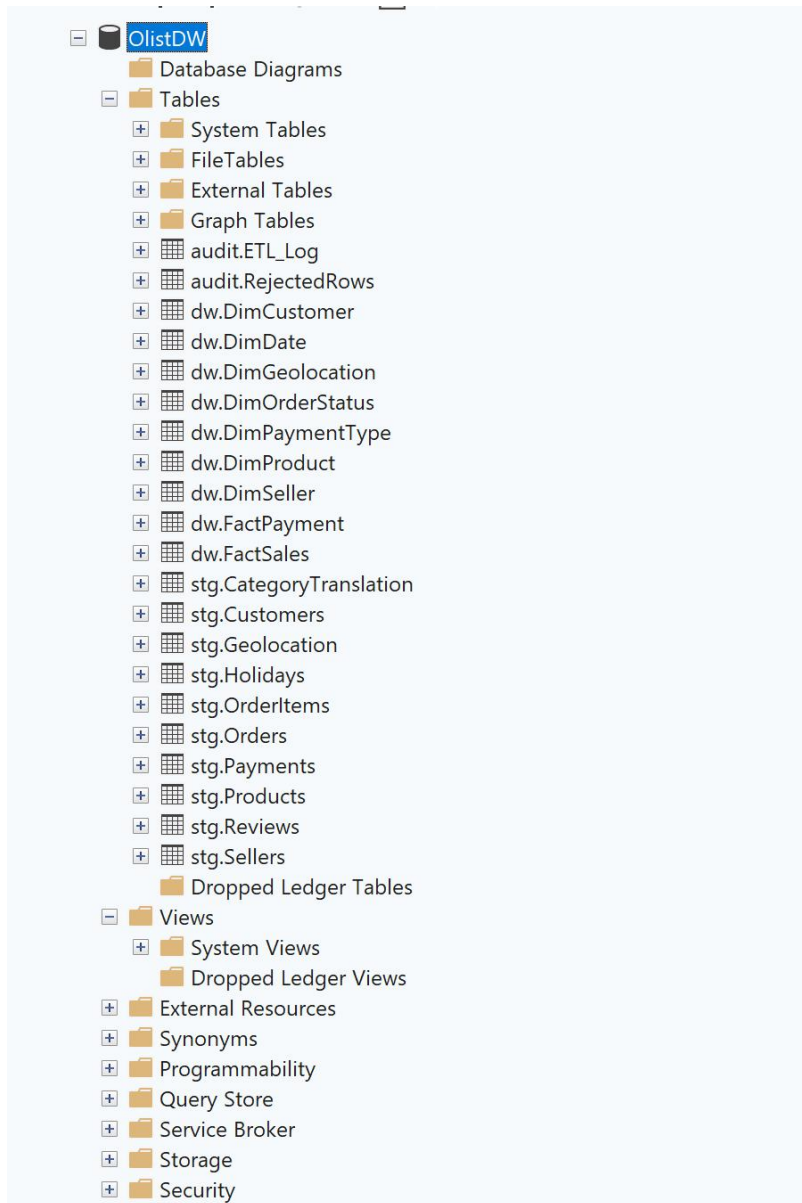


Figure: Star schema for the Olist data warehouse, showing FactSales and FactPayment sharing the DimDate, DimCustomer, and DimPaymentType dimensions. Each dimension includes an Unknown member row so that any unmatched sale or payment still joins correctly instead of being lost

A **star schema** (rather than snowflake) was chosen, since dimensions connect directly to the fact tables without further normalisation. This keeps queries simpler and faster for BI reporting, which is a priority for this solution's dashboard use case.

4.4 Implementation

The schema was implemented in a SQLite relational database (OlistDW.sqlite) using Python and pandas, as SQLite requires no server installation and ensured consistent results across all team members' machines regardless of their local SQL Server setup. Equivalent SQL Server DDL and ETL scripts (see Appendix C) were also developed to demonstrate the same star schema in an enterprise RDBMS environment, and were successfully executed to confirm the design translates correctly to SQL Server.



4.5 Design Assumptions

- **Type 1 (overwrite)** is used for all dimensions, since historical tracking of changes is not required for this analysis.

- **Payment data** is kept in its own fact table, FactPayment, rather than apportioned across order items, since it is recorded once per order in the source system.
- **DimDate** is pre-populated for the full date range covered by the dataset (2016–2018), plus one Unknown row for any missing dates, rather than derived row-by-row from order timestamps, which is standard practice for date dimensions.
- **is_holiday** in DimDate is populated using the external holiday API source identified in Task 2.
- **Region** (North, Northeast, Center-West, Southeast, South) is derived from each Brazilian state to support regional analysis.

Task 5 - ETL Process Development

5.1 Extract

Extraction method: Batch extraction using Python (pandas) and a custom script (`run_all.py`). Each source file was read fully into memory, validated, then passed into the transformation stage.

Source connection: The pipeline connected to five different source formats:

- CSV files (customers, orders, order_items, payments, reviews, products, sellers, geolocation)
- Excel (category_translation.xlsx)
- JSON/API (Brazilian public holidays, fetched live from the Nager.Date API — 42 holiday records for 2016–2018)
- Relational source (SQLite staging database built in Task 1)
- XML (product category metadata)

Data extraction process:

1. Each of the 9 Olist source tables was loaded using `pandas.read_csv()`.
2. The category translation table was read from Excel using `pandas.read_excel()`.
3. Holiday data was pulled via an HTTP GET request to the Nager.Date public API and parsed from JSON.
4. Row counts were checked immediately after extraction to confirm no files failed to load (e.g., geolocation: 1,000,163 rows extracted).

5.2 Transform

Transformation type	What was done
Missing value handling	Null order_delivered_customer_date values (2,965 orders, 2.98%) were flagged as "not yet delivered" rather than dropped
Duplicate removal	Geolocation table had 98.1% duplication by zip-code prefix; de-duplicated down to 19,015 unique zip prefixes
Data cleansing	Product category names cleaned and matched against the translation table; 2 unmatched categories (pc_gamer,

Transformation type	What was done
	portateis_cozinha_e_preparadores_de_alimentos) identified and handled
Standardisation	State codes standardised and mapped to 5 Brazilian regions (North, Northeast, Central-West, Southeast, South)
Data type conversion	7 date columns converted from string to datetime format
Aggregation	Multiple review scores per order averaged into a single order-level review score
Derived attribute creation	DeliveryDays, DelayDays, OnTimeFlag (derived from date differences); WeightClass (derived from product weight); Region (derived from customer/seller state)

5.3 Load

Transformed data was loaded into:

Dimension tables (7): DimDate, DimCustomer, DimProduct, DimSeller, DimGeolocation, DimPaymentType, DimOrderStatus — each assigned a surrogate key, with an "Unknown" member row added for handling missing foreign key matches.

Fact tables (2): FactSales (grain: one row per order item) and FactPayment (grain: one row per payment) — because payments in Olist are recorded at the order level, not the item level, so a single fact table couldn't represent both grains accurately.

ETL workflow (sequence): Extract → Clean/Transform → Build dimensions → Assign surrogate keys → Build fact tables using dimension lookups → Validate referential integrity → Load into OlistDW.sqlite.

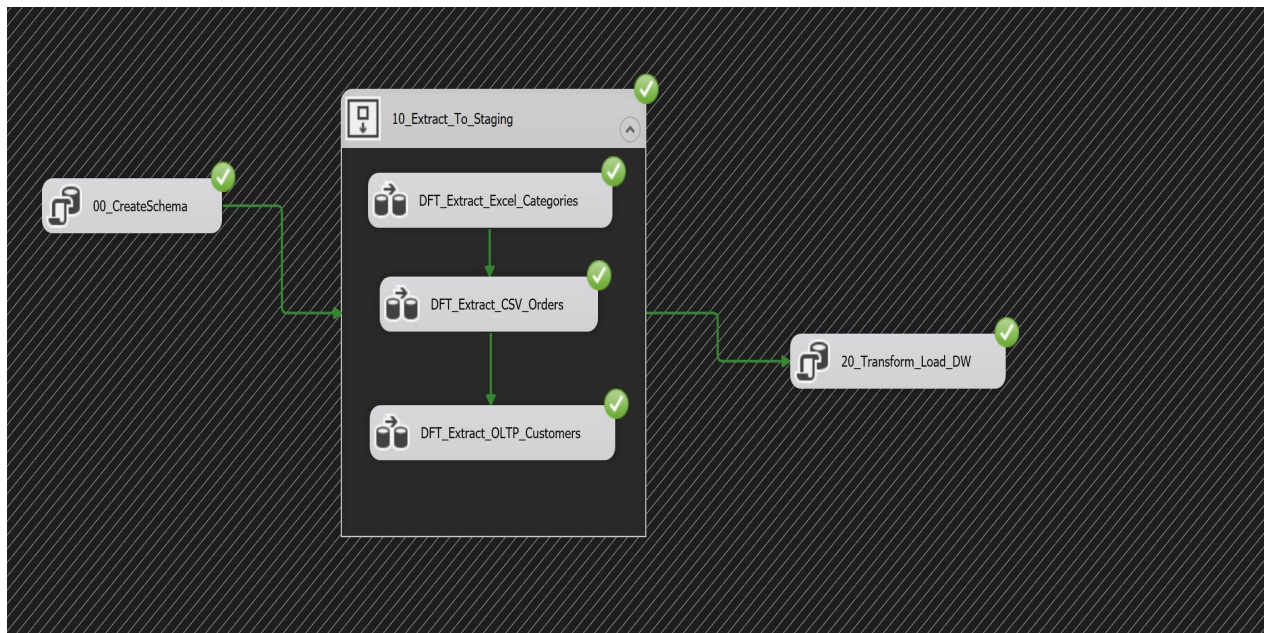
```
dim_product = pd.concat([unknown_p, dim_product], ignore_index=True)
Loaded DimDate          1,097 rows
Loaded DimCustomer      99,442 rows
Loaded DimProduct       32,952 rows
Loaded DimSeller        3,096 rows
Loaded DimGeolocation   19,015 rows
Loaded DimPaymentType    6 rows
Loaded DimOrderStatus    8 rows
Loaded FactSales        112,650 rows
Loaded FactPayment      103,886 rows
SQLite warehouse -> /Users/poorninakkavita/Downloads/DWBI-PROJECT/DWBI-PRO-Complete/OlistDW.sqlite
```

Validation results:

Table	Row count
-------	-----------

Table	Row count
DimDate	1,097
DimCustomer	99,442
DimProduct	32,952
DimSeller	3,096
DimGeolocation	19,015
DimPaymentType	6
DimOrderStatus	8
FactSales	112,650
FactPayment	103,886

Referential integrity check: **0 unmatched keys** across all fact-to-dimension relationships — confirming every fact row successfully matched a real dimension member (no rows fell back to "Unknown").



Task 6: Data Mart Development

6.1 Purpose of the Data Mart

A Delivery Performance data mart was created to help the logistics and operations team monitor delivery speed and reliability without needing to query the full data warehouse directly.

6.2 Target Users

Logistics managers and operations staff responsible for tracking seller and regional delivery performance.

6.3 Analytical Benefits

The mart pre-aggregates delivery metrics by product category, seller state, and customer region, producing 1,660 summary rows from the 112,650 individual order-item records in FactSales. This allows the operations team to quickly identify:

- Which categories or regions have the slowest average delivery times
- Which seller states have lower on-time delivery percentages
- Whether delivery speed correlates with customer review scores

without writing complex multi-table joins each time.

6.4 Implementation

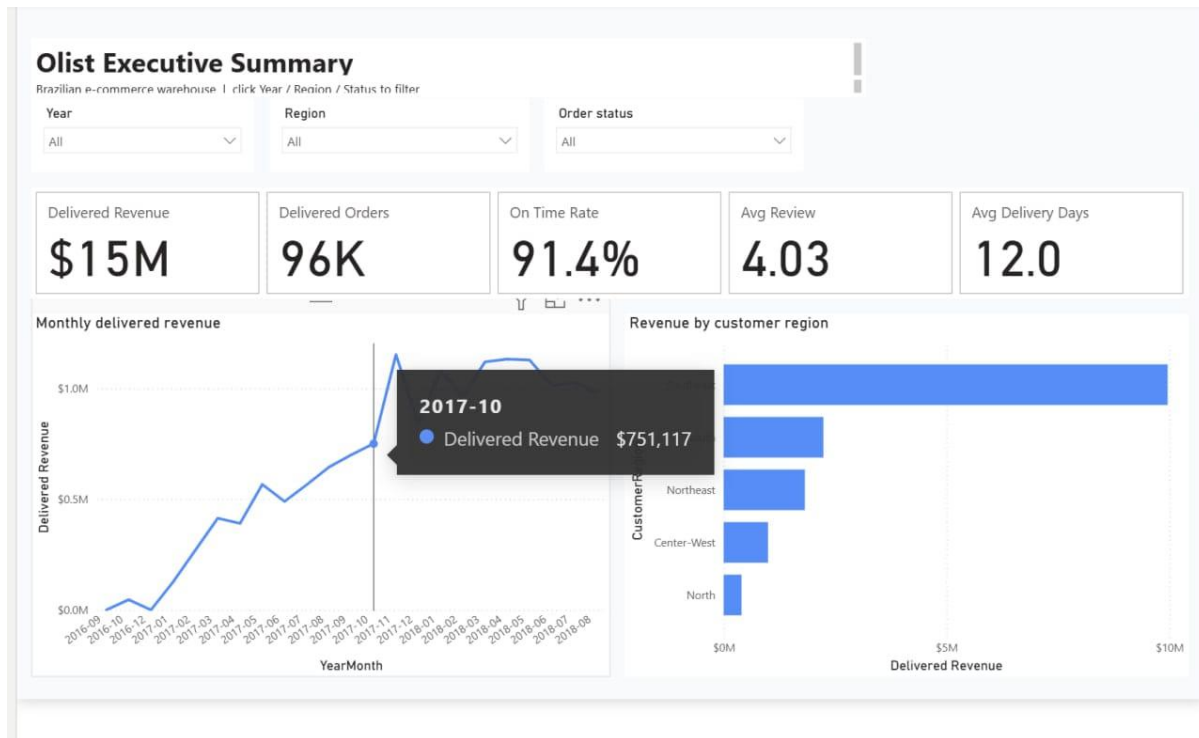
The mart was built as a summary table (DeliveryPerformanceMart) in the SQLite warehouse, aggregating FactSales by CategoryNameEnglish, SellerState, and CustomerRegion. Measures include TotalOrders, AvgDeliveryDays, OnTimePct, and AvgReviewScore. The mart was saved both as a database table and as a CSV export (08_GoldLayer/DeliveryPerformanceMart.csv) for direct use in the BI dashboard.

Data mart created: 1660 rows					
	Category	SellerState	CustomerRegion	TotalOrders	AvgDeliveryDays \
0	Other	DF	Center-West	1	15.000000
1	Other	DF	Northeast	2	19.000000
2	Other	DF	South	1	14.000000
3	Other	DF	Southeast	6	12.833333
4	Other	ES	North	1	26.000000
	OnTimePct		AvgReviewScore		
0	100.0		4.000000		
1	100.0		4.500000		
2	100.0		5.000000		
3	100.0		3.833333		
4	100.0		4.000000		

Task 7: OLAP Analysis and Business Intelligence Dashboard Development

An interactive Business Intelligence dashboard was developed using Microsoft Power BI[3] to analyse the Brazilian e-commerce dataset[1]. The dashboards use KPI cards, charts, slicers, and OLAP operations to provide different views of business performance.

Report Page 1: Executive Summary



The Executive Summary dashboard provides an overall view of business performance using key performance indicators and visualisations.

The dashboard includes the following KPIs:

- Delivered Revenue
- Delivered Orders
- On-Time Rate
- Average Review
- Average Delivery Days

A **Monthly Delivered Revenue** line chart is used to analyse revenue trends over time. A **Revenue by Customer Region** bar chart compares delivered revenue across different customer regions.

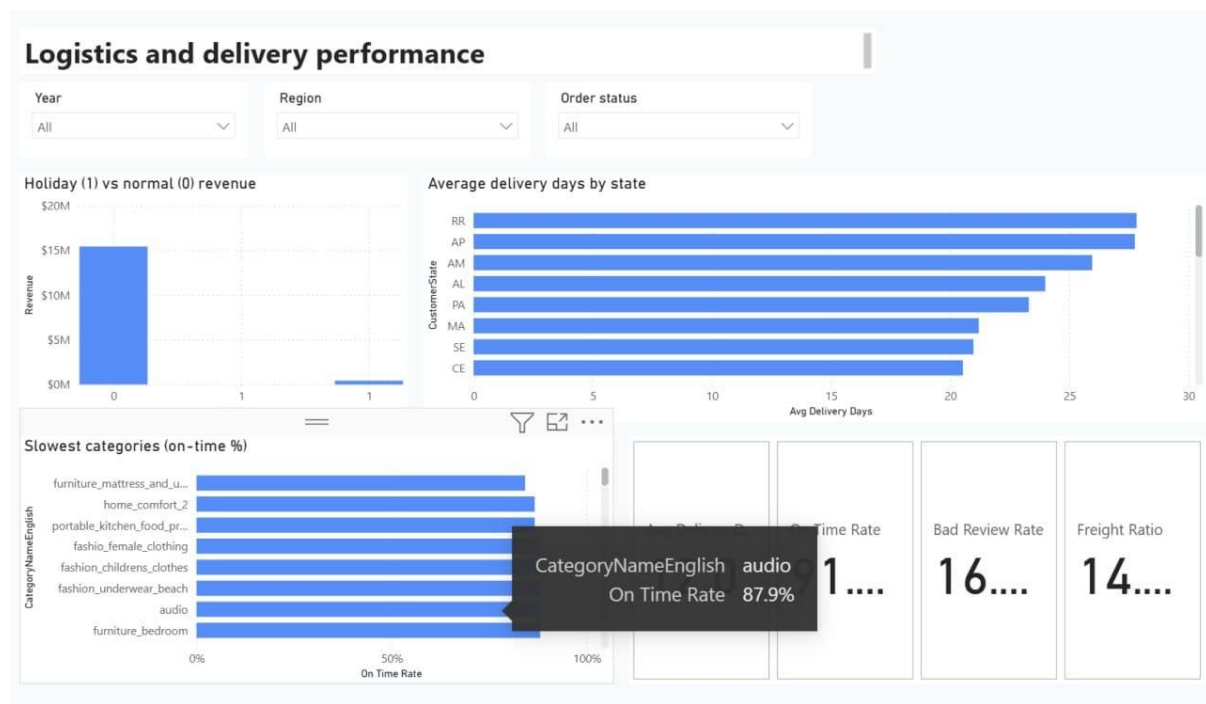
OLAP Operations:

- **Roll-up:** Monthly delivered revenue can be viewed as an aggregated view of overall revenue performance.

- **Drill-down:** The monthly revenue trend provides a more detailed time-based view compared with the overall revenue KPI.
- **Slice:** The data can be filtered by a selected **Year, Region, or Order Status**.
- **Dice:** Multiple slicers can be applied together, such as selecting a specific year, region, and order status.
- **Pivot:** Revenue can be viewed from different perspectives, such as comparing revenue across customer regions and across months.

The dashboard provides interactive slicers for **Year, Region, and Order Status**, allowing users to dynamically explore the business results.

Report Page 2: Logistics and Delivery Performance



The Logistics and Delivery Performance dashboard focuses on delivery efficiency and logistics-related business performance.

The dashboard includes a **Holiday vs Normal Revenue** comparison, an **Average Delivery Days by State** bar chart, and a **Slowest Categories by On-Time Rate** chart. It also includes KPI indicators such as **On-Time Rate, Bad Review Rate, and Freight Ratio**.

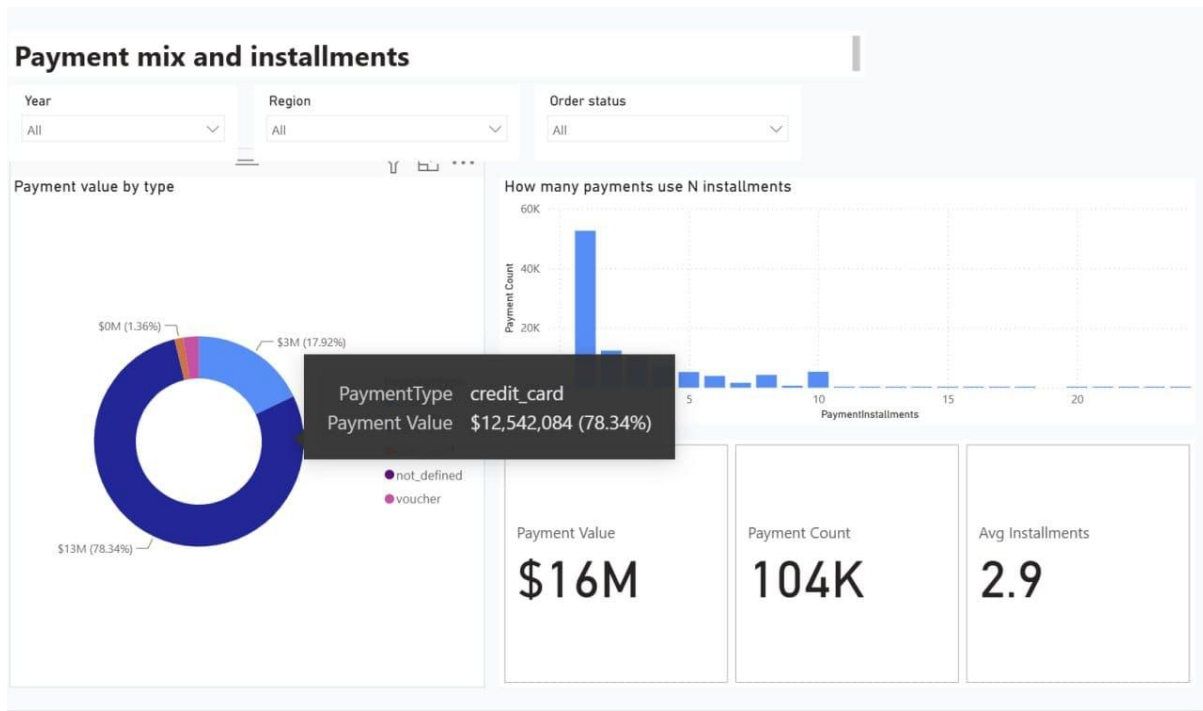
OLAP Operations:

- **Slice:** Users can select a particular year, region, or order status to analyse delivery performance for a specific subset of the data.
- **Dice:** Year, region, and order status can be combined to analyse delivery performance under multiple conditions.
- **Pivot:** Delivery performance can be examined from different perspectives, such as comparing **states by average delivery days** or **categories by on-time rate**.

- **Drill-down:** The state-level delivery chart provides a more detailed geographical view of delivery performance.
- **Roll-up:** State-level delivery information can be interpreted as an aggregated view of geographical delivery performance.

The interactive filters allow users to dynamically compare logistics performance across different business conditions.

Report Page 3: Payment Mix and Installments



The Payment Mix and Instalments dashboard analyses customer payment behaviour and instalment usage.

The dashboard includes a **Payment Value by Type** donut chart and a **Payments by Instalment Count** column chart. KPI cards display **Payment Value, Payment Count, and Average Instalments**.

OLAP Operations:

- **Slice:** Users can select a specific year, region, or order status to analyse payment behaviour for a particular subset.
- **Dice:** Multiple filters can be combined to analyse payment behaviour for a specific year, region, and order status simultaneously.
- **Pivot:** Payment information can be viewed from different perspectives, such as **payment type versus payment value** or **installment count versus payment count**.

- **Drill-down:** The installment chart allows payment behaviour to be examined at different installment counts rather than only as an overall payment total.
- **Roll-up:** Individual payment values and payment counts are aggregated to produce the overall **Payment Value** and **Payment Count** KPI values.

The dashboard therefore provides an interactive analysis of payment methods and installment patterns.

Summary of OLAP Operations

The three dashboards demonstrate the major OLAP operations used in Power BI:

OLAP Operation	Application in the Dashboard
Slice	Filtering data by Year, Region, or Order Status
Dice	Applying multiple filters simultaneously
Pivot	Viewing measures from different dimensions, such as region, state, category, payment type, or installments
Drill-down	Moving from aggregated information to more detailed time, geographical, category, or installment-level information
Roll-up	Aggregating detailed values into higher-level KPIs and summaries

Overall, the three dashboards provide interactive analysis of **business performance, logistics and delivery performance, and payment behaviour**. The combination of KPI cards, trend charts, comparison charts, slicers, and OLAP operations enable users to explore the e-commerce data from multiple perspectives and levels of detail.

Task 8: Business Insights and Recommendations

Insight 1: Revenue is heavily concentrated in the Southeast region

Evidence: The "Revenue by customer region" chart (Executive Summary page) shows the Southeast generating approximately \$9.96M of the \$15M total delivered revenue -around 65% of all sales while the North region contributes just over \$400K, the smallest share among all five regions.

Recommendation: Marketing and logistics investment should continue to prioritise the Southeast given its proven demand, but a targeted growth initiative (localized promotions, regional seller onboarding) should be piloted in the Centre-West and North regions, where revenue is disproportionately low relative to Brazil's population distribution.

Business decision supported: Regional budget allocation and market expansion planning.

Insight 2: Delivery times vary sharply by state, with remote states most affected

Evidence: The "Average delivery days by state" chart (Logistics and Delivery Performance page) shows states such as RR and AP averaging over 25 days for delivery, compared to closer to 20 days for states like CE and SE — a gap of 5+ days between the slowest and faster-served states.

Recommendation: Olist should evaluate partnering with regional logistics providers or establishing a distribution point in the North region to reduce delivery times to consistently underperforming states, rather than relying solely on centralized fulfilment from the Southeast.

Business decision supported: Logistics network and warehousing investment decisions.

Insight 3: Certain product categories consistently underperform on delivery reliability

Evidence: The "Slowest categories (on-time %)" chart shows the audio category with the lowest on-time delivery rate at 87.9%, alongside a below-average customer review score of 3.84, compared to top-performing categories exceeding 92% on-time delivery.

Recommendation: Investigate the audio category's fulfilment process specifically — this may point to a small number of underperforming sellers, oversized/fragile packaging causing delays, or sourcing from more distant regions. Sellers in this category should be flagged for a logistics review.

Business decision supported: Seller performance management and category-specific operational intervention.

Insight 4: Customers overwhelmingly pay by credit card, in a small number of instalments

Evidence: The "Payment value by type" chart shows credit card accounts for 78.34% of total payment value (\$12.5M of \$16M), with an average of 2.9 installments per transaction, while boleto (17.92%) and other methods make up a much smaller share.

Recommendation: Since credit card dominates payment behaviour, Olist should ensure credit card processing remains fast and reliable as a platform priority, and consider promotional financing (e.g. interest-free installments up to 3-4 months) to encourage higher-value purchases, given customers already show comfort with multi-installment payments.

Business decision supported: Payment partnership strategy and promotional/financing design.

Insight 5: Revenue is minimally affected by public holidays, suggesting steady non-holiday demand

Evidence: The "Holiday (1) vs normal (0) revenue" chart shows the vast majority of revenue — over \$15M — occurring on non-holiday days, with holiday-day revenue comprising only a small fraction of the total.

Recommendation: Rather than concentrating promotional campaigns around public holidays, Olist should maintain consistent marketing throughout regular business days, since the data shows demand is not holiday-driven. Marketing spend may be better allocated toward the seasonal spike observed in November (linked to Black Friday) rather than public holidays specifically.

Business decision supported: Marketing calendar and promotional campaign timing.

Summary Table

#	Insight	Recommendation
1	Southeast dominates revenue (65%)	Expand targeted marketing to underrepresented regions
2	North/remote states face longest delivery times	Invest in regional logistics partnerships
3	Audio category has lowest on-time rate (87.9%)	Review sellers/fulfilment for that category
4	Credit card dominates payments (78%), avg 2.9 installments	Prioritise card reliability; consider financing promotions
5	Holiday revenue is minimal vs regular days	Shift marketing focus away from holidays toward regular-season peaks (e.g. November)

References

- [1] Olist, "Brazilian E-Commerce Public Dataset by Olist," *Kaggle*, 2018. [Online]. Available: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>. [Accessed:18-09-2026].
- [2] Nager.Date, "Public Holiday API," [Online]. Available: <https://date.nager.at/>. [Accessed:22-092026].
- [3] Microsoft, "Power BI Documentation," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/power-bi/>. [Accessed:26-09-2026].

Appendix A - Attribute Summary

	table	attribute	dtype	null_count	null_pct
0	customers	customer_id	object	0	0.00
1	customers	customer_unique_id	object	0	0.00
2	customers	customer_zip_code_prefix	int64	0	0.00
3	customers	customer_city	object	0	0.00
4	customers	customer_state	object	0	0.00
5	orders	order_id	object	0	0.00
6	orders	customer_id	object	0	0.00
7	orders	order_status	object	0	0.00
8	orders	order_purchase_timestamp	object	0	0.00
9	orders	order_approved_at	object	160	0.16
10	orders	order_delivered_carrier_date	object	1783	1.79
11	orders	order_delivered_customer_date	object	2965	2.98
12	orders	order_estimated_delivery_date	object	0	0.00
13	order_items	order_id	object	0	0.00
14	order_items	order_item_id	int64	0	0.00
15	order_items	product_id	object	0	0.00
16	order_items	seller_id	object	0	0.00
17	order_items	shipping_limit_date	object	0	0.00
18	order_items	price	float64	0	0.00
19	order_items	freight_value	float64	0	0.00
20	payments	order_id	object	0	0.00
21	payments	payment_sequential	int64	0	0.00
22	payments	payment_type	object	0	0.00
23	payments	payment_installments	int64	0	0.00
24	payments	payment_value	float64	0	0.00

25	reviews	review_id	object	0	0.00
26	reviews	order_id	object	0	0.00
27	reviews	review_score	int64	0	0.00
28	reviews	review_comment_title	object	87656	88.34
29	reviews	review_comment_message	object	58247	58.70
30	reviews	review_creation_date	object	0	0.00
31	reviews	review_answer_timestamp	object	0	0.00
32	products	product_id	object	0	0.00
33	products	product_category_name	object	610	1.85
34	products	product_name_lenght	float64	610	1.85
35	products	product_description_lenght	float64	610	1.85
36	products	product_photos_qty	float64	610	1.85
37	products	product_weight_g	float64	2	0.01
38	products	product_length_cm	float64	2	0.01
39	products	product_height_cm	float64	2	0.01
40	products	product_width_cm	float64	2	0.01
41	sellers	seller_id	object	0	0.00
42	sellers	seller_zip_code_prefix	int64	0	0.00
43	sellers	seller_city	object	0	0.00
44	sellers	seller_state	object	0	0.00
45	geolocation	geolocation_zip_code_prefix	int64	0	0.00
46	geolocation	geolocation_lat	float64	0	0.00
47	geolocation	geolocation_lng	float64	0	0.00
48	geolocation	geolocation_city	object	0	0.00
49	geolocation	geolocation_state	object	0	0.00
50	category_translation	product_category_name	object	0	0.00
51	category_translation	product_category_name_english	object	0	0.00

Appendix B - Data Warehouse Schema (SQLite, built with Python/pandas)

DimDate

DateKey, FullDate, DayNumber, DayName, WeekdayFlag, WeekendFlag,
WeekOfYear, MonthNumber, MonthName, QuarterNumber, QuarterName,
YearNumber, YearMonth, IsHoliday, HolidayName

DimCustomer

CustomerKey, CustomerID, CustomerUniqueID, CustomerCity, CustomerState,
CustomerRegion, CustomerZipPrefix, IsUnknown

DimProduct

ProductKey, ProductID, CategoryNamePortuguese, CategoryNameEnglish,
WeightG, LengthCm, HeightCm, WidthCm, PhotosQty, WeightClass, IsUnknown

DimSeller

SellerKey, SellerID, SellerCity, SellerState, SellerRegion,
SellerZipPrefix, IsUnknown

DimGeolocation

GeoKey, ZipPrefix, City, State, Region, Latitude, Longitude

DimPaymentType

PaymentTypeKey, PaymentType, PaymentTypeGroup

DimOrderStatus

OrderStatusKey, OrderStatus, IsDeliveredFlag, IsCancelledFlag

FactSales

SalesKey, PurchaseDateKey, DeliveredDateKey, EstimatedDeliveryDateKey, CustomerKey, ProductKey, SellerKey, CustomerGeoKey, PaymentTypeKey, OrderStatusKey, OrderID, OrderItemID, Price, FreightValue, LineTotal, Quantity, DeliveryDays, DelayDays, OnTimeFlag, ReviewScore

FactPayment

PaymentKey, PurchaseDateKey, CustomerKey, PaymentTypeKey, OrderID, PaymentSequential, PaymentInstallments, PaymentValue

Each table was built as a pandas DataFrame and loaded into the SQLite database (OlistDW.sqlite) using `df.to_sql()`. Surrogate keys (e.g. CustomerKey, ProductKey) were generated as sequential integers, separate from the original source IDs, which are retained as attributes.

Appendix C — Equivalent SQL Server Implementation

The following SQL Server scripts implement the same star schema design shown in Section 4, demonstrating that the model translates correctly to an enterprise RDBMS platform. These scripts were executed successfully in SQL Server Management Studio (SSMS), creating the full OlistDW database schema (Section 4.4).

C.1 — Star Schema Creation (03_create_star_schema.sql)

```
03_create_star_schema.sql (SQL) X ^
4  */
5  USE OlistDW;
6  GO
7
8  IF OBJECT_ID(N'dw.FactSales', N'U') IS NOT NULL DROP TABLE dw.FactSales;
9  IF OBJECT_ID(N'dw.FactPayment', N'U') IS NOT NULL DROP TABLE dw.FactPayment;
10 IF OBJECT_ID(N'dw.DimDate', N'U') IS NOT NULL DROP TABLE dw.DimDate;
11 IF OBJECT_ID(N'dw.DimCustomer', N'U') IS NOT NULL DROP TABLE dw.DimCustomer;
12 IF OBJECT_ID(N'dw.DimProduct', N'U') IS NOT NULL DROP TABLE dw.DimProduct;
13 IF OBJECT_ID(N'dw.DimSeller', N'U') IS NOT NULL DROP TABLE dw.DimSeller;
14 IF OBJECT_ID(N'dw.DimGeolocation', N'U') IS NOT NULL DROP TABLE dw.DimGeolocation;
15 IF OBJECT_ID(N'dw.DimPaymentType', N'U') IS NOT NULL DROP TABLE dw.DimPaymentType;
16 IF OBJECT_ID(N'dw.DimOrderStatus', N'U') IS NOT NULL DROP TABLE dw.DimOrderStatus;
17 GO
18
19 CREATE TABLE dw.DimDate (
20     DateKey INT NOT NULL PRIMARY KEY,
21     FullDate DATE NOT NULL,
22     DayNumber TINYINT NOT NULL,
23     DayName VARCHAR(15) NOT NULL,
24     WeekdayFlag CHAR(1) NOT NULL,
25     WeekendFlag CHAR(1) NOT NULL,
26     WeekOfYear TINYINT NOT NULL,
27     MonthNumber TINYINT NOT NULL,
28     MonthName VARCHAR(15) NOT NULL,
29     QuarterNumber TINYINT NOT NULL,
30     QuarterName CHAR(2) NOT NULL,
31     YearNumber SMALLINT NOT NULL,
32     YearMonth CHAR(7) NOT NULL,
33     IsHoliday BIT NOT NULL DEFAULT 0,
34     HolidayName NVARCHAR(150) NULL
35 );
36
37 CREATE TABLE dw.DimCustomer (
38     CustomerKey INT IDENTITY(1,1) PRIMARY KEY,
39     CustomerID VARCHAR(50) NOT NULL,
40     CustomerUniqueID VARCHAR(50) NOT NULL,
41     CustomerCity VARCHAR(100) NULL,
42     CustomerState VARCHAR(10) NULL,
43     CustomerRegion VARCHAR(20) NULL,
44     CustomerZipPrefix INT NULL,
45     IsUnknown BIT NOT NULL DEFAULT 0
46 );
47
48 CREATE TABLE dw.DimProduct (
49     ProductKey INT IDENTITY(1,1) PRIMARY KEY,
50     ProductID VARCHAR(50) NOT NULL,
51     CategoryNamePortuguese VARCHAR(100) NULL,
52     CategoryNameEnglish VARCHAR(100) NULL,
53     WeightG FLOAT NULL,
54     LengthCm FLOAT NULL,
55     HeightCm FLOAT NULL,
56     WidthCm FLOAT NULL,
57     PhotosQty INT NULL,
58     WeightClass VARCHAR(20) NULL,
59     IsUnknown BIT NOT NULL DEFAULT 0
60 );
61
62 CREATE TABLE dw.DimSeller (
63     SellerKey INT IDENTITY(1,1) PRIMARY KEY,
64     SellerID VARCHAR(50) NOT NULL,
65     SellerCity VARCHAR(100) NULL,
66     SellerState VARCHAR(10) NULL,
67     SellerRegion VARCHAR(20) NULL,
68     SellerZipPrefix INT NULL,
69     IsUnknown BIT NOT NULL DEFAULT 0
70 );
71
72 CREATE TABLE dw.DimGeolocation (
73     GeoKey INT IDENTITY(1,1) PRIMARY KEY,
74     ZipPrefix INT NOT NULL,
75     City VARCHAR(100) NULL,
76     State VARCHAR(10) NULL,
77     Region VARCHAR(20) NULL,
78     Latitude FLOAT NULL,
79     Longitude FLOAT NULL
80 );
81
82 Messages
83 Commands completed successfully.
84 Completion time: 2026-09-29T14:08:11.4907901+08:00
```

```
36
37 CREATE TABLE dw.DimCustomer (
38     CustomerKey INT IDENTITY(1,1) PRIMARY KEY,
39     CustomerID VARCHAR(50) NOT NULL,
40     CustomerUniqueID VARCHAR(50) NOT NULL,
41     CustomerCity VARCHAR(100) NULL,
42     CustomerState VARCHAR(10) NULL,
43     CustomerRegion VARCHAR(20) NULL,
44     CustomerZipPrefix INT NULL,
45     IsUnknown BIT NOT NULL DEFAULT 0
46 );
47
48 CREATE TABLE dw.DimProduct (
49     ProductKey INT IDENTITY(1,1) PRIMARY KEY,
50     ProductID VARCHAR(50) NOT NULL,
51     CategoryNamePortuguese VARCHAR(100) NULL,
52     CategoryNameEnglish VARCHAR(100) NULL,
53     WeightG FLOAT NULL,
54     LengthCm FLOAT NULL,
55     HeightCm FLOAT NULL,
56     WidthCm FLOAT NULL,
57     PhotosQty INT NULL,
58     WeightClass VARCHAR(20) NULL,
59     IsUnknown BIT NOT NULL DEFAULT 0
60 );
61
62 CREATE TABLE dw.DimSeller (
63     SellerKey INT IDENTITY(1,1) PRIMARY KEY,
64     SellerID VARCHAR(50) NOT NULL,
65     SellerCity VARCHAR(100) NULL,
66     SellerState VARCHAR(10) NULL,
67     SellerRegion VARCHAR(20) NULL,
68     SellerZipPrefix INT NULL,
69     IsUnknown BIT NOT NULL DEFAULT 0
70 );
71
72 CREATE TABLE dw.DimGeolocation (
73     GeoKey INT IDENTITY(1,1) PRIMARY KEY,
74     ZipPrefix INT NOT NULL,
75     City VARCHAR(100) NULL,
76     State VARCHAR(10) NULL,
77     Region VARCHAR(20) NULL,
78     Latitude FLOAT NULL,
79     Longitude FLOAT NULL
80 );
81
82 Messages
83 Commands completed successfully.
84 Completion time: 2026-09-29T14:08:11.4907901+08:00
```

```

81
82 CREATE TABLE dw.DimPaymentType (
83     PaymentTypeKey INT IDENTITY(1,1) PRIMARY KEY,
84     PaymentType VARCHAR(30) NOT NULL,
85     PaymentTypeGroup VARCHAR(30) NOT NULL
86 );
87
88 CREATE TABLE dw.DimOrderStatus (
89     OrderStatusKey INT IDENTITY(1,1) PRIMARY KEY,
90     OrderStatus VARCHAR(30) NOT NULL,
91     IsDeliveredFlag CHAR(1) NOT NULL,
92     IsCancelledFlag CHAR(1) NOT NULL
93 );
94
95 CREATE TABLE dw.FactSales (
96     SalesKey BIGINT IDENTITY(1,1) PRIMARY KEY,
97     PurchaseDateKey INT NOT NULL,
98     DeliveredDateKey INT NULL,
99     EstimatedDeliveryDateKey INT NULL,
100     CustomerKey INT NOT NULL,
101     ProductKey INT NOT NULL,
102     SellerKey INT NOT NULL,
103     CustomerGeoKey INT NULL,
104     PaymentTypeKey INT NULL,
105     OrderStatusKey INT NOT NULL,
106     OrderID VARCHAR(50) NOT NULL, -- degenerate
107     OrderItemID INT NOT NULL, -- degenerate
108     Price DECIMAL(12,2) NOT NULL,
109     FreightValue DECIMAL(12,2) NOT NULL,
110     LineTotal DECIMAL(12,2) NOT NULL,
111     Quantity INT NOT NULL DEFAULT 1,
112     DeliveryDays INT NULL,
113     DelayDays INT NULL,
114     OnTimeFlag CHAR(1) NULL,
115     ReviewScore DECIMAL(4,2) NULL,
116     CONSTRAINT FK_FactSales_PurchaseDate FOREIGN KEY (PurchaseDateKey) REFERENCES dw.DimDate(DateKey),
117     CONSTRAINT FK_FactSales_Customer FOREIGN KEY (CustomerKey) REFERENCES dw.DimCustomer(CustomerKey),
118     CONSTRAINT FK_FactSales_Product FOREIGN KEY (ProductKey) REFERENCES dw.DimProduct(ProductKey),
119     CONSTRAINT FK_FactSales_Seller FOREIGN KEY (SellerKey) REFERENCES dw.DimSeller(SellerKey),
120     CONSTRAINT FK_FactSales_Status FOREIGN KEY (OrderStatusKey) REFERENCES dw.DimOrderStatus(OrderStatusKey)
121 );
122
123 CREATE TABLE dw.FactPayment (
124     PaymentKey BIGINT IDENTITY(1,1) PRIMARY KEY,
125     PurchaseDateKey INT NOT NULL,
126     CustomerKey INT NOT NULL,
127     PaymentTypeKey INT NOT NULL,
128     OrderID VARCHAR(50) NOT NULL,
129     PaymentSequential INT NOT NULL,
130     PaymentInstallments INT NOT NULL,
131     PaymentValue DECIMAL(12,2) NOT NULL,
132     CONSTRAINT FK_FactPayment_Date FOREIGN KEY (PurchaseDateKey) REFERENCES dw.DimDate(DateKey),
133     CONSTRAINT FK_FactPayment_Cust FOREIGN KEY (CustomerKey) REFERENCES dw.DimCustomer(CustomerKey),
134     CONSTRAINT FK_FactPayment_Type FOREIGN KEY (PaymentTypeKey) REFERENCES dw.DimPaymentType(PaymentTypeKey)
135 );
136 GO
137
138 CREATE UNIQUE INDEX IX_DimCustomer_BK ON dw.DimCustomer(CustomerID);
139 CREATE UNIQUE INDEX IX_DimProduct_BK ON dw.DimProduct(ProductID);
140 CREATE UNIQUE INDEX IX_DimSeller_BK ON dw.DimSeller(SellerID);
141 CREATE UNIQUE INDEX IX_DimGeo_Zip ON dw.DimGeolocation(ZipPrefix);
142 CREATE UNIQUE INDEX IX_DimPayType ON dw.DimPaymentType(PaymentType);
143 CREATE UNIQUE INDEX IX_DimStatus ON dw.DimOrderStatus(OrderStatus);
144 CREATE INDEX IX_FactSales_Date ON dw.FactSales(PurchaseDateKey);
145 CREATE INDEX IX_FactSales_Product ON dw.FactSales(ProductKey);
146 CREATE INDEX IX_FactSales_Customer ON dw.FactSales(CustomerKey);
147 CREATE INDEX IX_FactSales_Order ON dw.FactSales(OrderID);
148 GO
149

```

C.2 — ETL Load Procedures (04_etl_load_dw.sql)

```
7  USE OlistDW;
8  GO
9
10 CREATE OR ALTER PROCEDURE dw.usp_LoadDimensions
11 AS
12 BEGIN
13     SET NOCOUNT ON;
14
15     /* ----- DimDate ----- */
16     TRUNCATE TABLE dw.FactPayment;
17     TRUNCATE TABLE dw.FactSales;
18
19     DELETE FROM dw.DimCustomer;
20     DELETE FROM dw.DimProduct;
21     DELETE FROM dw.DimSeller;
22     DELETE FROM dw.DimGeolocation;
23     DELETE FROM dw.DimPaymentType;
24     DELETE FROM dw.DimOrderStatus;
25     DELETE FROM dw.DimDate;
26
27     IF EXISTS (SELECT 1 FROM sys.identity_columns WHERE object_id = OBJECT_ID('dw.DimCustomer') AND last_value IS NOT NULL)
28         DBCC CHECKIDENT ('dw.DimCustomer', RESEED, 0);
29     IF EXISTS (SELECT 1 FROM sys.identity_columns WHERE object_id = OBJECT_ID('dw.DimProduct') AND last_value IS NOT NULL)
30         DBCC CHECKIDENT ('dw.DimProduct', RESEED, 0);
31     IF EXISTS (SELECT 1 FROM sys.identity_columns WHERE object_id = OBJECT_ID('dw.DimSeller') AND last_value IS NOT NULL)
32         DBCC CHECKIDENT ('dw.DimSeller', RESEED, 0);
33
34     DECLARE @Start DATE = '2010-01-01';
35     DECLARE @End   DATE = '2010-12-31';
36
37     ;WITH Dates AS (
38         SELECT @Start AS d
39         UNION ALL
40         SELECT DATEADD(DAY, 1, d) FROM Dates WHERE d < @End
41     )
42     INSERT INTO dw.DimDate (
43         DateKey, FullDate, DayNumber, DayName, WeekdayFlag, WeekendFlag,
44         WeekOfYear, MonthNumber, MonthName, QuarterNumber, QuarterName,
45         YearNumber, YearMonth, IsHoliday, HolidayName
46     )
47     SELECT
48         YEAR(d) * 10000 + MONTH(d) * 100 + DAY(d),
49         d,
50         DAY(d),
51         DATENAME(WEEKDAY, d),
52         CASE WHEN DATEPART(WEEKDAY, d) IN (1, 7) THEN 'N' ELSE 'Y' END,
53         CASE WHEN DATEPART(WEEKDAY, d) IN (1, 7) THEN 'Y' ELSE 'N' END,
54         DATEPART(ISO_WEEK, d),
55         MONTH(d),
56         DATENAME(MONTH, d),
57         DATENAME(QUARTER, d),
58         'Q' + CAST(DATEPART(QUARTER, d) AS CHAR(1)),
59         YEAR(d),
60         FORMAT(d, 'yyyy-MM'),
61         CASE WHEN h.holiday_date IS NULL THEN 0 ELSE 1 END,
62         h.english_name
63     FROM Dates
64     LEFT JOIN (
65         SELECT holiday_date, MAX(english_name) AS english_name
66         FROM stg.Holidays
67         GROUP BY holiday_date
68     ) h ON h.holiday_date = Dates.d
69     OPTION (MAXRECURSION 0);
70
71     INSERT INTO dw.DimDate (
72         DateKey, FullDate, DayNumber, DayName, WeekdayFlag, WeekendFlag,
73         WeekOfYear, MonthNumber, MonthName, QuarterNumber, QuarterName,
74         YearNumber, YearMonth, IsHoliday, HolidayName
75     )
76     SELECT 19000101, '1900-01-01', 1, 'Monday', 'Y', 'N', 1, 1, 'January', 1, 'Q1', 1900, '1900-01', 0, NULL
77     WHERE NOT EXISTS (SELECT 1 FROM dw.DimDate WHERE DateKey = 19000101);
78
79     /* ----- DimPaymentType ----- */
80     INSERT INTO dw.DimPaymentType (PaymentType, PaymentTypeGroup)
81     SELECT DISTINCT
82         ISNULL(NULLIF(LTRIM(RTRIM(payment_type)), ''), 'not_defined'),
83         CASE
84             WHEN payment_type IN ('credit_card', 'debit_card') THEN 'Card'
85             WHEN payment_type = 'invoice' THEN 'Bank slip'
86             WHEN payment_type = 'voucher' THEN 'Voucher'
87             ELSE 'Other'
88         END
89     FROM stg.Payments;
90
91     IF NOT EXISTS (SELECT 1 FROM dw.DimPaymentType WHERE PaymentType = 'unknown')
92         INSERT INTO dw.DimPaymentType (PaymentType, PaymentTypeGroup) VALUES ('unknown', 'Other');
93
94     /* ----- DimOrderStatus ----- */
95     INSERT INTO dw.DimOrderStatus (OrderStatus, IsDeliveredFlag, IsCancelledFlag)
96     SELECT DISTINCT
97         order_status,
98         CASE WHEN order_status = 'delivered' THEN 'Y' ELSE 'N' END,
99         CASE WHEN order_status IN ('canceled', 'cancelled') THEN 'Y' ELSE 'N' END
100     FROM stg.Orders;
101
102     /* ----- DimGeolocation (deduplicate zip prefixes) ----- */
103     INSERT INTO dw.DimGeolocation (ZipPrefix, City, State, Region, Latitude, Longitude)
104     SELECT
105         geolocation_zip_code_prefix,
106         MAX(geolocation_city),
107         MAX(geolocation_state),
108         CASE MAX(geolocation_state)
109             WHEN 'AC' THEN 'North' WHEN 'AD' THEN 'North' WHEN 'AM' THEN 'North'
110             WHEN 'PA' THEN 'North' WHEN 'RO' THEN 'North' WHEN 'RR' THEN 'North' WHEN 'TO' THEN 'North'
111             WHEN 'AL' THEN 'Northeast' WHEN 'BA' THEN 'Northeast' WHEN 'CE' THEN 'Northeast'
112             WHEN 'PA' THEN 'Northeast' WHEN 'PB' THEN 'Northeast' WHEN 'PE' THEN 'Northeast'
113             WHEN 'PI' THEN 'Northeast' WHEN 'RN' THEN 'Northeast' WHEN 'SE' THEN 'Northeast'
114             WHEN 'DF' THEN 'Center-West' WHEN 'GO' THEN 'Center-West'
115             WHEN 'MT' THEN 'Center-West' WHEN 'MS' THEN 'Center-West'
116             WHEN 'ES' THEN 'Southeast' WHEN 'MG' THEN 'Southeast'
117             WHEN 'RJ' THEN 'Southeast' WHEN 'SP' THEN 'Southeast'
118             WHEN 'PR' THEN 'South' WHEN 'RS' THEN 'South' WHEN 'SC' THEN 'South'
119             ELSE 'Unknown'
120         END,
121         AVG(geolocation_lat),
122         AVG(geolocation_lng)
123     FROM
124         stg.Geolocation
125     GROUP BY geolocation_zip_code_prefix;
126
127     /* ----- Unknown members ----- */
128     SET IDENTITY_INSERT dw.DimCustomer ON;
129     INSERT INTO dw.DimCustomer (CustomerKey, CustomerID, CustomerUniqueID, CustomerCity, CustomerState, CustomerRegion, CustomerZipPrefix, IsUnknown)
130     VALUES (-1, 'UNKNOWN', 'UNKNOWN', 'Unknown', 'XX', 'Unknown', 0, 1);
131     SET IDENTITY_INSERT dw.DimCustomer OFF;
```

```

INSERT INTO dw.DimCustomer (CustomerID, CustomerUniqueID, CustomerCity, CustomerState, CustomerRegion, CustomerZipPrefix, IsUnknown)
SELECT
    c.customer_id,
    c.customer_unique_id,
    LOWER(LTRIM(RTRIM(c.customer_city))),
    UPPER(LTRIM(RTRIM(c.customer_state))),
    CASE UPPER(LTRIM(RTRIM(c.customer_state)))
        WHEN 'AC' THEN 'North' WHEN 'AD' THEN 'North' WHEN 'AM' THEN 'North'
        WHEN 'PA' THEN 'North' WHEN 'RO' THEN 'North' WHEN 'RR' THEN 'North' WHEN 'TO' THEN 'North'
        WHEN 'AL' THEN 'Northeast' WHEN 'BA' THEN 'Northeast' WHEN 'CE' THEN 'Northeast'
        WHEN 'MA' THEN 'Northeast' WHEN 'PB' THEN 'Northeast' WHEN 'PE' THEN 'Northeast'
        WHEN 'PI' THEN 'Northeast' WHEN 'RN' THEN 'Northeast' WHEN 'SE' THEN 'Northeast'
        WHEN 'DF' THEN 'Center-West' WHEN 'GO' THEN 'Center-West'
        WHEN 'MT' THEN 'Center-West' WHEN 'MS' THEN 'Center-West'
        WHEN 'ES' THEN 'Southeast' WHEN 'MS' THEN 'Southeast'
        WHEN 'RJ' THEN 'Southeast' WHEN 'SP' THEN 'Southeast'
        WHEN 'PR' THEN 'South' WHEN 'RS' THEN 'South' WHEN 'SC' THEN 'South'
        ELSE 'Unknown'
    END,
    c.customer_zip_code_prefix,
    0
FROM stg.Customers c;

SET IDENTITY_INSERT dw.DimProduct ON;
INSERT INTO dw.DimProduct (ProductKey, ProductID, CategoryNamePortuguese, CategoryNameEnglish, WeightClass, IsUnknown)
VALUES (-1, 'UNKNOWN', 'Unknown', 'Unknown', 'Unknown', 1);
SET IDENTITY_INSERT dw.DimProduct OFF;

INSERT INTO dw.DimProduct (
    ProductID, CategoryNamePortuguese, CategoryNameEnglish,
    WeightG, LengthCm, HeightCm, WidthCm, PhotosQty, WeightClass, IsUnknown
)
SELECT
    p.product_id,
    ISNULL(p.product_category_name, 'Unknown'),
    ISNULL(c.product_category_name_english,
        CASE p.product_category_name
            WHEN 'pc-games' THEN 'pc-games'
            WHEN 'portateis-cozinha_e_preparadores_de_alimentos' THEN 'portable_kitchen_food_preparers'
            ELSE 'Other'
        END),
    p.product_weight_g,
    p.product_length_cm,
    p.product_height_cm,
    p.product_width_cm,
    p.product_photos_qty,
    CASE
        WHEN p.product_weight_g IS NULL THEN 'Unknown'
        WHEN p.product_weight_g < 500 THEN 'Light'
        WHEN p.product_weight_g < 2000 THEN 'Medium'
        ELSE 'Heavy'
    END,
    0
FROM stg.Products p
LEFT JOIN stg.CategoryTranslation t
ON p.product_category_name = t.product_category_name;

SET IDENTITY_INSERT dw.DimSeller ON;
INSERT INTO dw.DimSeller (SellerKey, SellerID, SellerCity, SellerState, SellerRegion, SellerZipPrefix, IsUnknown)
VALUES (-1, 'UNKNOWN', 'Unknown', 'XX', 'Unknown', 0, 1);
SET IDENTITY_INSERT dw.DimSeller OFF;

INSERT INTO dw.DimSeller (SellerID, SellerCity, SellerState, SellerRegion, SellerZipPrefix, IsUnknown)
SELECT
    s.seller_id,
    LOWER(LTRIM(RTRIM(s.seller_city))),
    UPPER(LTRIM(RTRIM(s.seller_state))),
    CASE UPPER(LTRIM(RTRIM(s.seller_state)))
        WHEN 'AC' THEN 'North' WHEN 'AP' THEN 'North' WHEN 'AM' THEN 'North'
        WHEN 'PA' THEN 'North' WHEN 'RO' THEN 'North' WHEN 'RR' THEN 'North' WHEN 'TO' THEN 'North'
        WHEN 'AL' THEN 'Northeast' WHEN 'BA' THEN 'Northeast' WHEN 'CE' THEN 'Northeast'
        WHEN 'MA' THEN 'Northeast' WHEN 'PB' THEN 'Northeast' WHEN 'PE' THEN 'Northeast'
        WHEN 'PI' THEN 'Northeast' WHEN 'RN' THEN 'Northeast' WHEN 'SE' THEN 'Northeast'
        WHEN 'DF' THEN 'Center-West' WHEN 'GO' THEN 'Center-West'
        WHEN 'MT' THEN 'Center-West' WHEN 'MS' THEN 'Center-West'
        WHEN 'ES' THEN 'Southeast' WHEN 'MS' THEN 'Southeast'
        WHEN 'RJ' THEN 'Southeast' WHEN 'SP' THEN 'Southeast'
        WHEN 'PR' THEN 'South' WHEN 'RS' THEN 'South' WHEN 'SC' THEN 'South'
        ELSE 'Unknown'
    END,
    s.seller_zip_code_prefix,
    0
FROM stg.Sellers s;

END;
GO

CREATE OR ALTER PROCEDURE dw.usp_LoadFacts
AS
BEGIN
    SET NOCOUNT ON;

    ;WITH PrimaryPay AS (
        SELECT order_id, payment_type,
            ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY payment_sequential) AS rn
        FROM stg.Payments
    ),
    AvgReview AS (
        SELECT order_id, AVG(CAST(review_score AS DECIMAL(4,2))) AS review_score
        FROM stg.Reviews
        GROUP BY order_id
    )
    INSERT INTO dw.FactSales (
        PurchaseDateKey, DeliveredDateKey, EstimatedDeliveryDateKey,
        CustomerKey, ProductKey, SellerKey, CustomerGeoKey, PaymentTypeKey, OrderStatusKey,
        OrderID, OrderItemID, Price, FreightValue, LineTotal, Quantity,
        DeliveryDays, DelayDays, OnTimeFlag, ReviewScore
    )
    SELECT
        ISNULL(dd.p.DateKey, 19000101),
        dd.d.DateKey,
        dd.e.DateKey,
        ISNULL(c.CustomerKey, -1),
        ISNULL(p.ProductKey, -1),
        ISNULL(s.SellerKey, -1),
        g.GeoKey,
        ISNULL(pt.PaymentTypeKey, (SELECT TOP 1 PaymentTypeKey FROM dw.DimPaymentType WHERE PaymentType = 'unknown')),
        os.OrderStatusKey,
        i.order_id,
        i.order_item_id,
        i.price,
        i.freight_value,
        i.price + i.freight_value,
        1,
        CASE WHEN o.order_delivered_customer_date IS NULL THEN NULL
            ELSE DATEDIFF(DAY, o.order_purchase_timestamp, o.order_delivered_customer_date) END,

```

```

        ELSE DATEDIFF(DAY, o.order_purchase_timestamp, o.order_delivered_customer_date) END,
CASE WHEN o.order_delivered_customer_date IS NULL OR o.order_estimated_delivery_date IS NULL THEN NULL
      ELSE DATEDIFF(DAY, o.order_estimated_delivery_date, o.order_delivered_customer_date) END,
CASE
  WHEN o.order_delivered_customer_date IS NULL OR o.order_estimated_delivery_date IS NULL THEN NULL
  WHEN o.order_delivered_customer_date <= o.order_estimated_delivery_date THEN 'Y'
  ELSE 'N'
END,
r.review_score
FROM stg.OrderItems i
INNER JOIN stg.Orders o ON o.order_id = i.order_id
LEFT JOIN dw.DimCustomer c ON c.CustomerID = o.customer_id
LEFT JOIN dw.DimProduct p ON p.ProductID = i.product_id
LEFT JOIN dw.DimSeller s ON s.SellerID = i.seller_id
LEFT JOIN dw.DimOrderStatus os ON os.OrderStatus = o.order_status
LEFT JOIN dw.DimDate dd_p ON dd_p.FullDate = CAST(o.order_purchase_timestamp AS DATE)
LEFT JOIN dw.DimDate dd_d ON dd_d.FullDate = CAST(o.order_delivered_customer_date AS DATE)
LEFT JOIN dw.DimDate dd_e ON dd_e.FullDate = CAST(o.order_estimated_delivery_date AS DATE)
LEFT JOIN dw.DimGeolocation g ON g.ZipPrefix = c.CustomerZipPrefix
LEFT JOIN PrimaryPay pay ON pay.order_id = i.order_id AND pay.rn = 1
LEFT JOIN dw.DimPaymentType pt ON pt.PaymentType = pay.payment_type
LEFT JOIN AvgReview r ON r.order_id = i.order_id;

INSERT INTO dw.FactPayment (
  PurchaseDateKey, CustomerKey, PaymentTypeKey, OrderID,
  PaymentSequential, PaymentInstallments, PaymentValue
)
SELECT
  ISNULL(dd.DateKey, 19900101),
  ISNULL(c.CustomerKey, -1),
  ISNULL(pt.PaymentTypeKey, (SELECT TOP 1 PaymentTypeKey FROM dw.DimPaymentType WHERE PaymentType = 'unknown')),
  pay.order_id,
  pay.payment_sequential,
  pay.payment_installments,
  pay.payment_value
FROM stg.Payments pay
INNER JOIN stg.Orders o ON o.order_id = pay.order_id
LEFT JOIN dw.DimCustomer c ON c.CustomerID = o.customer_id
LEFT JOIN dw.DimPaymentType pt ON pt.PaymentType = pay.payment_type
LEFT JOIN dw.DimDate dd ON dd.FullDate = CAST(o.order_purchase_timestamp AS DATE);
END;
GO

```